TECHNICAL DOCUMENTATION

SQL

https://github.com/mailen-cotignola

Contents

Accommodation Management System Inspired by Peer-to-Peer Rental Platforms 2
- Introduction 2
- Objectives 2
- Business Model 2
- Conceptual ER Diagram 3
- Schematic ER Diagram 4
- Database Table Descriptions 4
- Views 9
- Functions 10
- Stored Procedures 10
- Triggers 11
- Business Intelligence (BI) 11
- Technologies Used 13

Accommodation Management System Inspired by Peer-to-Peer Rental Platforms

Introduction

In today's digital era, the temporary accommodation industry has transformed radically. Platforms such as Airbnb have changed how people search, book, and share homes with travelers worldwide. These platforms democratize travel by offering diverse options—from cozy city apartments to remote countryside houses.

The project aims to deliver a comprehensive and reliable lodging-management solution inspired by Airbnb. Using an efficient database, the goal is to manage bookings and properties easily to improve the experience for both owners and travelers.

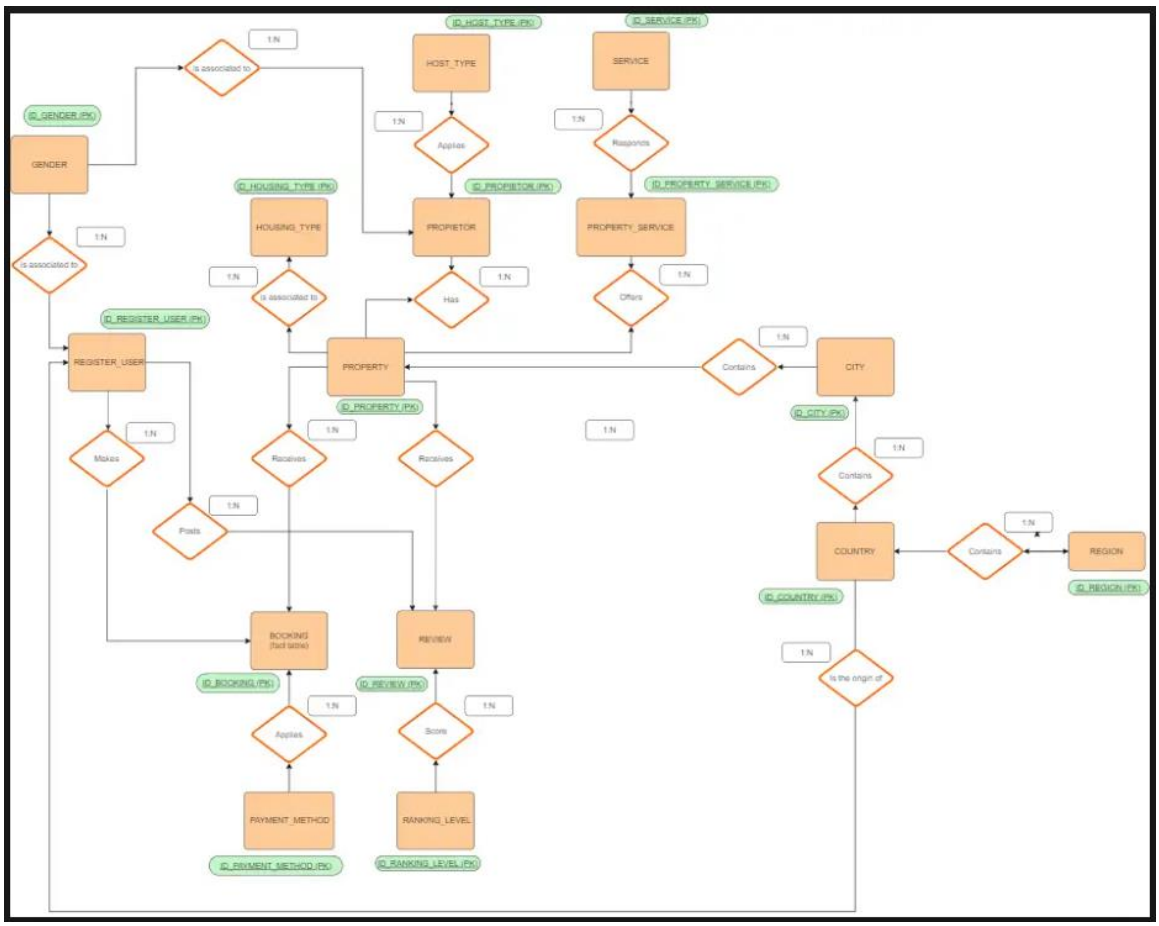
Objectives

- Market analysis of short-term rentals: demand, supply, prices, and trends.
- Identify needs and opportunities for investors and interested users.

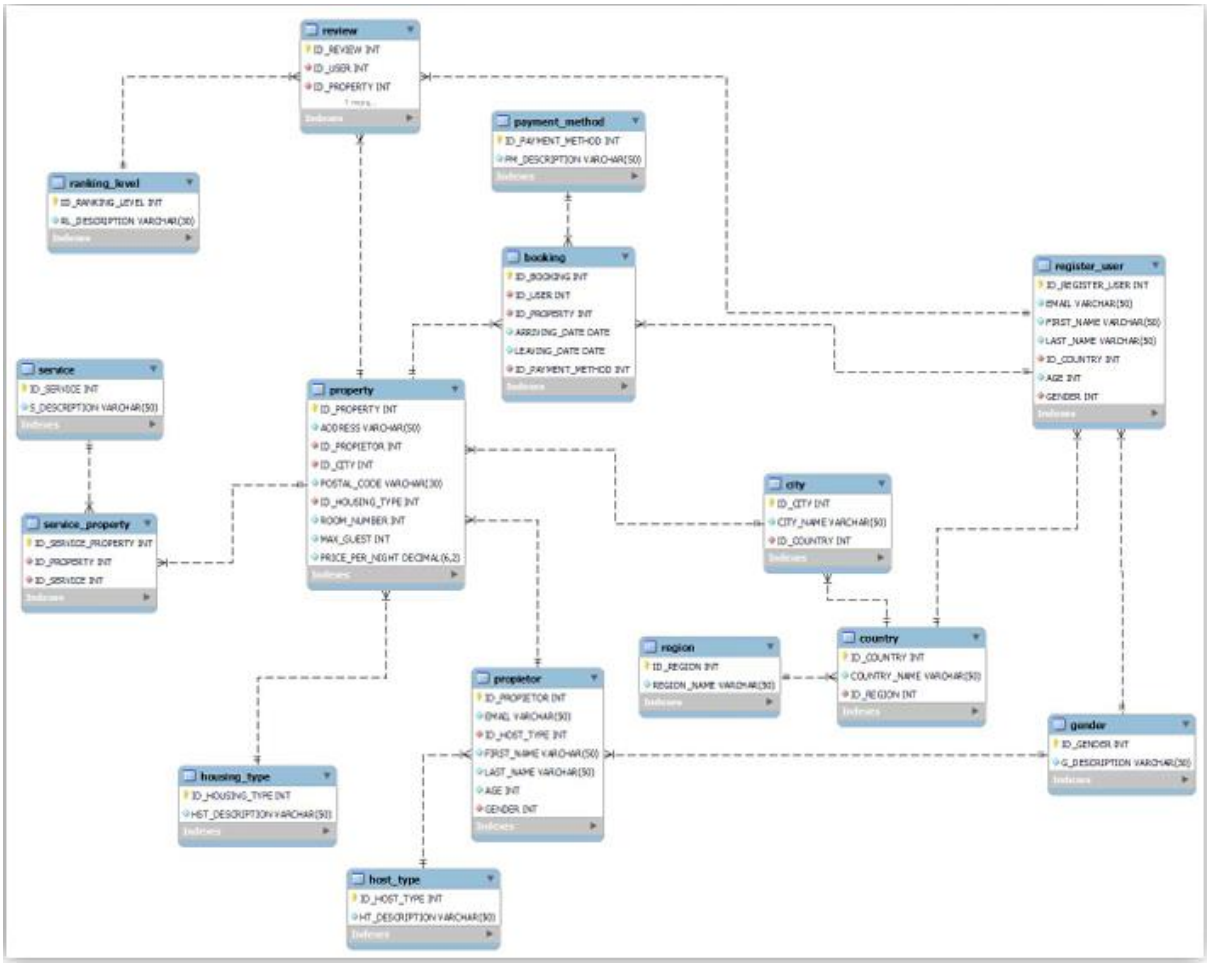
Business Model

The project follows a peer-to-peer model, where users book temporary stays and experiences without intermediaries.

Conceptual ER Diagram



Schematic ER Diagram



Database Table Descriptions

• HOST_TYPE: Dimensional. It contains different categories from the owner/proprietor (beginner, super host, expert, etc.).

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_HOST_TYPE	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	HT_DESCRIPTION	VARCHAR	30	NOT NULL	UNIQUE	

• REGION: Dimensional. Geographic region of the property.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_REGION	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	REGION_NAME	VARCHAR	50	NOT NULL	UNIQUE	

• COUNTRY: Dimensional. Country where the property is located.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_COUNTRY	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	COUNTRY_NAME	VARCHAR	50	NOT NULL	UNIQUE	
FK	ID_REGION	INT		NOT NULL	NO	

• CITY: Dimensional. City where the property is located.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_CITY	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	CITY_NAME	VARCHAR	50	NOT NULL	UNIQUE	
FK	ID_COUNTRY	INT		NOT NULL	NO	

• RANKING_LEVEL: Dimensional. Rating levels users assign to hosts.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_RANKING_LEVEL	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	RL_DESCRIPTION	VARCHAR	30	NOT NULL	UNIQUE	

• GENDER: Dimensional. Gender of users and hosts.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_GENDER	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	G_DESCRIPTION	VARCHAR	15	NOT NULL	UNIQUE	

• PROPIETOR: Dimensional. Core data of property owners.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_PROPIETOR	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	EMAIL	VARCHAR	50	NOT NULL	UNIQUE	
FK	ID_HOST_TYPE	INT		NOT NULL	NO	
	FIRST_NAME	VARCHAR	50	NOT NULL	NO	
	LAST_NAME	VARCHAR	50	NOT NULL	NO	
	AGE	INT		NOT NULL	NO	
FK	GENDER	INT		NOT NULL	NO	

• HOUSING_TYPE: Dimensional. Type of building or lodging.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_HOUSING_TYPE	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	HST_DESCRIPTION	VARCHAR	50	NOT NULL	UNIQUE	

• PROPERTY: Dimensional. Description of each property.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_PROPERTY	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
UNI	ADDRESS	VARCHAR	50	NOT NULL	UNIQUE	
FK	ID_PROPIETOR	INT		NOT NULL	NO	
FK	ID_CITY	INT		NOT NULL	NO	
MULL	POSTAL_CODE	VARCHAR	30	NOT NULL	NO	
FK	ID_HOUSING_TYPE	INT		NOT NULL	NO	
	ROOM_NUMBER	INT		NOT NULL	NO	
	MAX_GUEST	INT		NOT NULL	NO	
	PRICE_PER_NIGHT	DECIMAL		NOT NULL	NO	

• REGISTER_USER: Dimensional. Registered users and their basic information.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_REGISTER_USER	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
UNI	EMAIL	VARCHAR	50	NOT NULL	UNIQUE	
	FIRST_NAME	VARCHAR	50	NOT NULL	NO	
	LAST_NAME	VARCHAR	50	NOT NULL	NO	
FK	ID_COUNTRY	INT		NOT NULL	NO	
	AGE	INT		NOT NULL	NO	
FK	GENDER	INT		NOT NULL	NO	

• PAYMENT_METHOD: Dimensional. User payment methods.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_PAYMENT_METHOD	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	PM_DESCRIPTION	VARCHAR	50	NOT NULL	UNIQUE	

• BOOKING: Fact table. Details of each reservation.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_BOOKING	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
FK	ID_USER	INT		NOT NULL	NO	
FK	ID_PROPERTY	INT		NOT NULL	NO	
	ARRIVING_DATE	DATE		NOT NULL	NO	
	LEAVING_DATE	DATE		NOT NULL	NO	
FK	ID_PAYMENT_METHOD	INT		NOT NULL	NO	

• REVIEW: Dimensional. Ratings users assign to hosts.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_REVIEW	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
FK	ID_USER	INT		NOT NULL	NO	
FK	ID_PROPERTY	INT		NOT NULL	NO	
FK	ID_RANKING_LEVEL	INT		NOT NULL	NO	

• SERVICE: Dimensional. Available property services.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_SERVICE	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
	S_DESCRIPTION	VARCHAR	50	NOT NULL	UNIQUE	

• SERVICE_PROPERTY: Dimensional. Relations between properties and services.

KEY	COLUMN	TYPE	LENGHT	NOT NULL	UNIQUE	DEFAULT
PK	ID_SERVICE_PROPERTY	INT		NOT NULL	UNIQUE	AUTO_INCREMENT
FK	ID_PROPERTY	INT		NOT NULL	NO	
FK	ID_SERVICE	INT		NOT NULL	NO	
FK	ID_RANKING_LEVEL	INT		NOT NULL	NO	

Views

1. VW_best_rated: Shows the accommodations highest rated by guests (ranking level = 4 or higher) in descending order. It includes users responsible for the rating by ID, the ranking received, and the property it refers to, therefore, the table involved is REVIEW. This way, we can know which properties provide the greatest satisfaction to users.
2. VW_best_rated_address: Same as above but adds city/address. This makes it more transparent to visualize the property's location/address in a compact way, not just with ID numbers. The tables involved are CITY and PROPERTY.
3. VW_total_days: View of the total days of each booking; the goal is to show the calculation of days that the booking lasted, based on its arrival and departure dates. The tables involved are REGISTER_USER and BOOKING.
4. VW_total_income: Show the total amount of money received for each booking. This will inform us of the total income of each booking with the relevant data for it. The tables involved are PROPERTY and BOOKING.
5. VW_long_stay: Show the names of the users and their country of origin for those who used long-stay bookings (>60 days). This will allow us to have a better understanding of market trends, and whether short-term or long-term accommodation is chosen. The tables involved are REGISTER_USER and COUNTRY.

Functions

1. get_average_rating_by_country: This function could calculate and return the average rating of a country. It considers the ratings received for the properties located there. The tables involved are REVIEW, PROPERTY, CITY, and COUNTRY. With this, we can have a better idea of how satisfied customers are based on the properties offered by each country.
2. count_properties_in_city: By entering the name of a specific city, we can see the number of properties located there. The tables involved are PROPERTY and CITY.

Stored Procedures

- sp_get_propietor_names_order: Sort the "propietor" table according to a sorting field and direction. This stored procedure is used to sort the "propietor" table in the database based on a specified sorting field and a sorting direction (ascending or descending) provided as parameters. It allows customizing the sorting of the "propietor" table according to different criteria.

  Objective: to provide a flexible way to sort the "propietor" table according to a specific field and a specific sorting direction. It allows users to sort the table according to their needs and obtain the results in the desired order.

  Parameter details:
  a. order_column (CHAR(20)): The field by which to sort the "propietor" table.
  b. order_direction (CHAR(4)): The desired sorting direction ("ASC" for ascending, "DESC" for descending).

- create_booking: Inserts a new booking with validations. Allows storing records in the booking table of the database. Its main purpose is to provide a structured and centralized way to insert new reservations into the database, ensuring that certain conditions and validations are met.

  Objective: to provide a secure and efficient mechanism to insert new reservations into the booking table. By centralizing the insertion logic in a stored procedure, we ensure that the proper rules and validations are followed before making changes to the database. This promotes data consistency and facilitates the maintenance and management of booking operations.

  Parameter details:
  a. Id_user: The ID of the user making the reservation.
  b. Id_property: The ID of the property being reserved.
  c. arriving_date: The guest's arrival date.

d. leaving_date: The guest's departure date.
e. Id_payment_method: The ID of the selected payment method.

Triggers

- price_per_night_register: The purpose of this trigger is to keep a record of changes in the "price_per_night" field of the "property" table. Whenever the nightly price of a property is updated, a new record will be inserted into the "price_per_night_register" table with the property ID, the previous price, the new price, the user making the update, and the date and time of the operation.
  By using this trigger, it is possible to track changes in the nightly price of properties and maintain a historical record of the modifications made. For example, as these records accumulate, they will allow us to identify trends in price increases, which cities experienced the most dramatic price changes, and by what percentage. This would be a type of analysis that could be applied using this information.

  Type: BEFORE

  Monitors: UPDATE

  Table: property

- after_propietor_update_email: The purpose of this trigger is to keep a record of changes in the "email" field of the "propietor" table. Each time a proprietor's email is updated, a new record will be inserted into the "propietor_update_email" table with the proprietor's ID, the previous email, the new email, and the date of the change.
  By using this trigger, it is possible to track changes in the proprietors' emails and have a historical record of the modifications made. Therefore, in case it is necessary to contact the proprietor at some point, and the email address cannot be found, we could confirm if it was changed and when.

  Type: AFTER

  Monitors: UPDATE

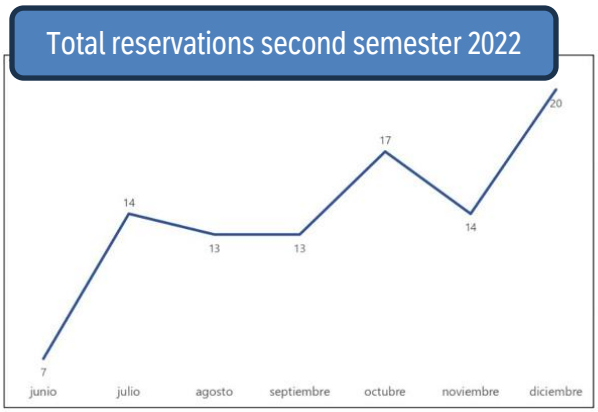
  Table: propietor

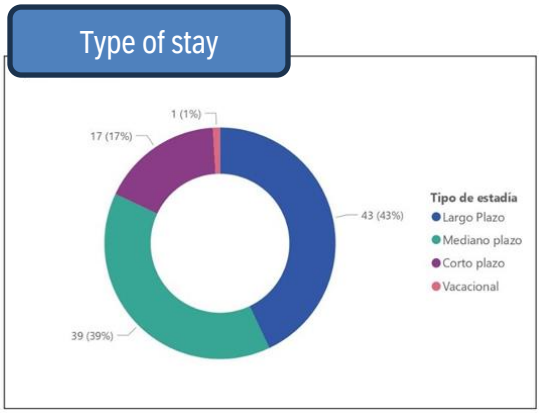
Business Intelligence (BI)

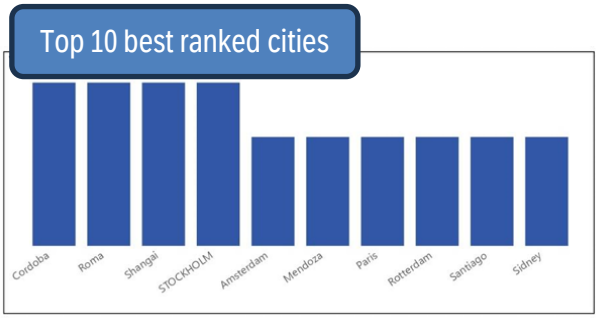
Using the views and tables generated in the database, we can create useful charts and dashboards.

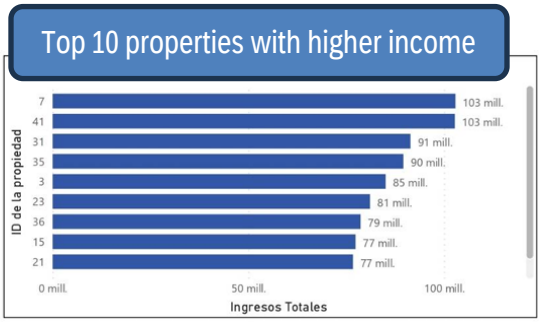
From this, the resulting charts are:

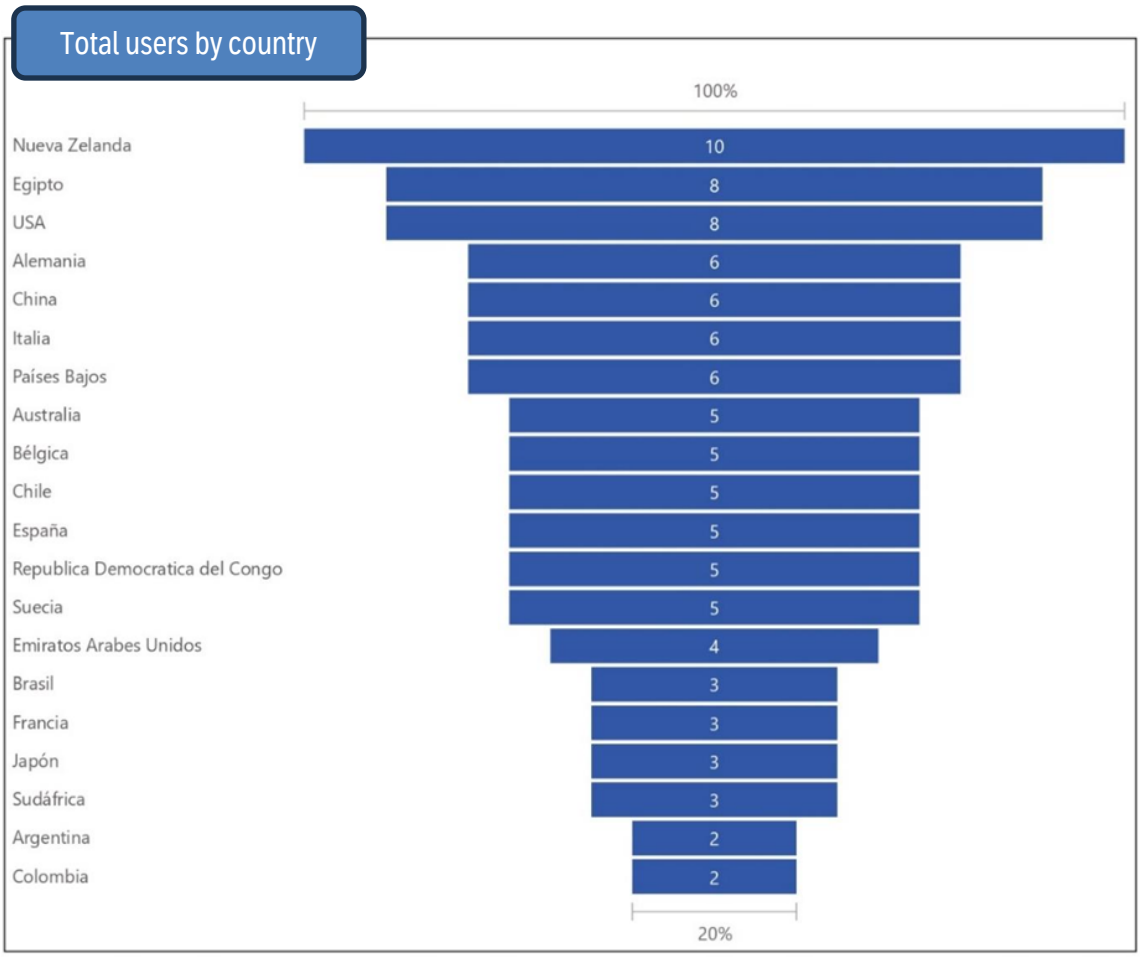
- Reservation trends over time.
- Stay types based on total reserved days.
- Best-rated cities.
- Properties with highest income.
- Distribution of users by country.











Technologies Used

System	Product	Version
Database	MySQL Community Server - GPL	8.0.33 for Win64 on x86_64
Draw.io	DER Diagrams	v21.6.7
BI	Microsoft Power BI Desktop	2.99.782.0 64-bit